

Two-Stage Colour Sorting System

ENGE X200 Microcontroller Programming – Assignment 1 Report

Hiten Gupta (a1918705) · Adelaide University · 25 September 2026

Appendix

#	Document / file	What it is	Used in
A1	This report (PDF)	Explains the code logic, register initialisation, ISRs, flowcharts, test results and discussion.	All sections
A2	docs/final.drawio.pdf (source: final.drawio)	Master flowchart. Shows power-up, the full <i>init()</i> register initialisation, the FSM state variables, and the <i>main()</i> loop that runs the output actions for each state (IDLE, ITEM_NEW, STAGE1, STAGE2, FAULT).	Section 5, Figure 1
A3	docs/interrupts.drawio.pdf (source: interrupts.drawio)	ISR flowcharts. One separate flowchart per IRQ handler: <i>EXTI2_3_IRQHandler</i> (New Item, PA2) and <i>EXTI4_15_IRQHandler</i> (Step Gate PA10 and E-Stop PB5 share this vector).	Section 5, Figure 2
A4	docs/state.drawio.pdf (source: state.drawio)	Finite-state-machine diagram. Shows the five states, the button that causes each transition, and the gate/LED outputs in each state.	Section 5, Figure 3
A5	docs/data.txt	Raw test log written while testing in Wokwi: real colour, sensed colour, Gate 1 and Gate 2 action for 30 items.	Section 6, Table 3
A6	Video.mp4 (docs/Video)	Screen recording of the code running in Wokwi. Please ignore the last part of the video (from about 4:40 to the end at 5:27): the recording did not stop properly, so this part only shows unrelated screen activity.	Section 6
A7	Source code	Src/main.c, Src/assignment1/assignment_1.h, my_helpers.c/.h, Src/stm_helper/my_stm_helper.c/.h, startup, linker script, CMakeLists.txt, cmake/, Inc/drivers, diagram.json, wokwi.toml. Everything needed to build in VS Code and run in Wokwi.	Sections 3, 4, 8

Report contents: 1 Introduction · 2 Hardware Configuration · 3 Register Initialisation · 4 Code Logic, ISRs and Subroutines · 5 Flowcharts · 6 Test Results · 7 Discussion of Correctness · 8 Acknowledgement of Lecture Code · 9 Conclusion

1. Introduction

This report describes embedded C code for a two-stage sorting system on the STM32C031C6.

New Item creates an item whose real and sensed colours are shown on two RGB LEDs, and **Step Gate** moves two servo gates through the sorting sequence. Gate 1 sends **sensed** blue items 45° anticlockwise (ACW) to the blue bin and sends green/red items 45° clockwise (CW) to stage 2. Gate 2 moves 45° ACW for green, 45° CW for red, and stays neutral for blue. The **E-Stop** button turns on the Fault LED and returns both gates to neutral. All actions are started by hardware interrupts.

2. Hardware Configuration

Signal	Pin	Configuration
New Item	PA2	Input, pull-down, EXTI line 2 (rising) → EXTI2_3_IRQHandler
Step Gate	PA10	Input, pull-down, EXTI line 10 (rising) → EXTI4_15_IRQHandler
E-Stop	PB5	Input, pull-down, EXTI line 5 (rising) → EXTI4_15_IRQHandler
Gate 1 servo	PC7	Alternate function AF1 = TIM3_CH2, PWM 50 Hz
Gate 2 servo	PB0	Alternate function AF1 = TIM3_CH3, PWM 50 Hz
Fault LED	PA9	Output
Real colour RGB (R, G, B)	PA0, PA1, PA4	Outputs
Sensed colour RGB (R, G, B)	PA3, PA11, PA8	Outputs

3. Register Initialisation – init()

init() is called once at the start of *main()* and configures every peripheral in this order:

1. **Clocks (RCC).** *Clock_Enable()* sets RCC→IOPENR bits 0–2 (GPIOA, GPIOB, GPIOC) and RCC→APBENR1 bit 1 (TIM3).
2. **Inputs.** PA2, PA10 and PB5 are set to MODER = 00 (input) with PUPDR = 10 (pull-down) (idle LOW, rising edge on press).
3. **Outputs.** PA9, PA0, PA1, PA4, PA3, PA11 and PA8 are set to MODER = 01 (general-purpose output).
4. **Servo pins.** PC7 and PB0 are set to MODER = 10 (alternate function) and *GPIO_ConnectPeripheral()* writes AF1 into the AFR register, connecting them to TIM3_CH2 and TIM3_CH3.
5. **TIM3 time base.** *Timer_Init(TIM3, ..., 47, 19999, ...)* sets PSC = 47 and ARR = 19999, enables auto-reload preload (ARPE), and sets the UG bit to load these values straight away. With $f_{clk} = 48 \text{ MHz}$: $f_{PWM} = 48 \text{ MHz} / ((47+1)(19999+1)) = 50 \text{ Hz}$ (20 ms period). Each timer tick is 1 μs , so CCR equals the pulse width in microseconds.
6. **PWM channels.** *PWM_Init()* sets CCxS = 00 (output), OCxM = 110 (PWM mode 1) and OCxPE (CCR preload) in CCMR1 (CH2) and CCMR2 (CH3), enables the outputs in CCER (bits 4 and 8), and starts the counter (CR1.CEN). Both gates are then set to CCR = 1500 (neutral).
7. **EXTI.** *EXTI_Init()* writes the port number into EXTICR[pin/4] at bit (pin%4)×8 (PA2 → EXTICR[0], PB5 → EXTICR[1] = 0x01, PA10 → EXTICR[2]), selects the rising edge in RTSR1, unmask the line in IMR1, clears any stale pending flag and enables EXTI2_3_IRQn / EXTI4_15_IRQn in the NVIC.
8. **SysTick.** *SysTick_Init()* (provided code) starts the 24-bit SysTick counter, which *Generate_Next_Item()* uses as a random seed.

Servo angles. Using the lecture servo mapping $t_n = 500 + (2000/180) \cdot \theta \mu\text{s}$, the three gate positions are defined in my_helpers.h as GATE_ANGLE_AC = 1000 μs (45°, i.e. 45° ACW of

neutral), GATE_ANGLE_NEUTRAL = 1500 μ s (90°) and GATE_ANGLE_C = 2000 μ s (135°, 45° CW of neutral).

4. Code Logic, ISRs and Subroutines

4.1 Finite state machine

As suggested in the brief and Lecture 7, the sorting logic is a finite state machine using *typedef enum SystemState_t* and a switch-case. The variable *current_state* (volatile, starts as STATE_IDLE) only changes through *changeState()*, which also sets the *isStateChange* flag.

State	Entered by	Outputs
STATE_IDLE	Power-up, or 3rd Step Gate press	Real and Sensed LEDs off, Fault LED off, both gates 90°
STATE_ITEM_NEW	New Item (only allowed from IDLE)	Generate_Next_Item() lights the Real and Sensed LEDs; both gates 90°
STATE_STAGE1	1st Step Gate press	Gate 1: 45° ACW if sensed BLUE, otherwise 45° CW
STATE_STAGE2	2nd Step Gate press	Gate 2: 45° ACW if sensed GREEN, 45° CW if RED, neutral if BLUE
STATE_FAULT	E-Stop from any state	Fault LED on, both gates 90°; no exit (simulation must be restarted)

Table 2 – FSM states and their outputs

4.2 Interrupt service routines

EXTI2_3_IRQHandler (New Item, PA2): checks the rising pending bit RPR1 bit 2. If the system is in STATE_IDLE it changes to STATE_ITEM_NEW. It then clears the flag by writing 1 to RPR1 bit 2. This blocks a new item until the previous one has finished both stages.

EXTI4_15_IRQHandler (Step Gate PA10 and E-Stop PB5): both pins use the same interrupt vector, so the handler checks each pending bit separately. For bit 10 (Step Gate), a switch-case moves ITEM_NEW → STAGE1 → STAGE2 → IDLE. In IDLE or FAULT the Step Gate press is ignored, so the gates cannot move without an item. For bit 5 (E-Stop), the state is set to STATE_FAULT whatever the current state. Each flag is cleared after it has been handled.

4.3 Main loop

After *init()*, *main()* enters *while(1)*. When an ISR has changed the state (*isStateChange* = true), the loop clears the flag and runs the switch-case output actions for the new state once (Table 2). The gate decisions use *Read_Sensor_RGB()*, so the gates always act on the colour shown on the Sensed LED.

4.4 Custom subroutines (my_helpers.c)

- **Set_Truth_RGB() / Set_Sensor_RGB():** required by the provided code. Using the ItemColour_t value, they set exactly one of the R/G/B pins HIGH and clear the other two. OFF clears all three.
- **Pin_Set_If():** small helper that sets or clears a pin depending on a condition, so the colour code has no repeated if-else blocks.
- **Set_Fault():** drives the Fault LED (PA9).
- **Read_Truth_RGB() / Read_Sensor_RGB():** read back the LED pins through IDR and return the matching ItemColour_t.
- **Gate1_SetWidth() / Gate2_SetWidth():** write the pulse width in μs to TIM3→CCR2 (PC7) and TIM3→CCR3 (PB0). CCR preload applies the new width at the next 20 ms period.

5. Flowcharts

All diagrams were made in draw.io (software, not hand-drawn). The ISRs could not all fit on one diagram, so Figure 1 is the master flowchart, Figure 2 has a separate flowchart for each IRQ handler, and Figure 3 is the state diagram that links them. The original PDFs are submitted in the docs folder (Appendix A2–A4).

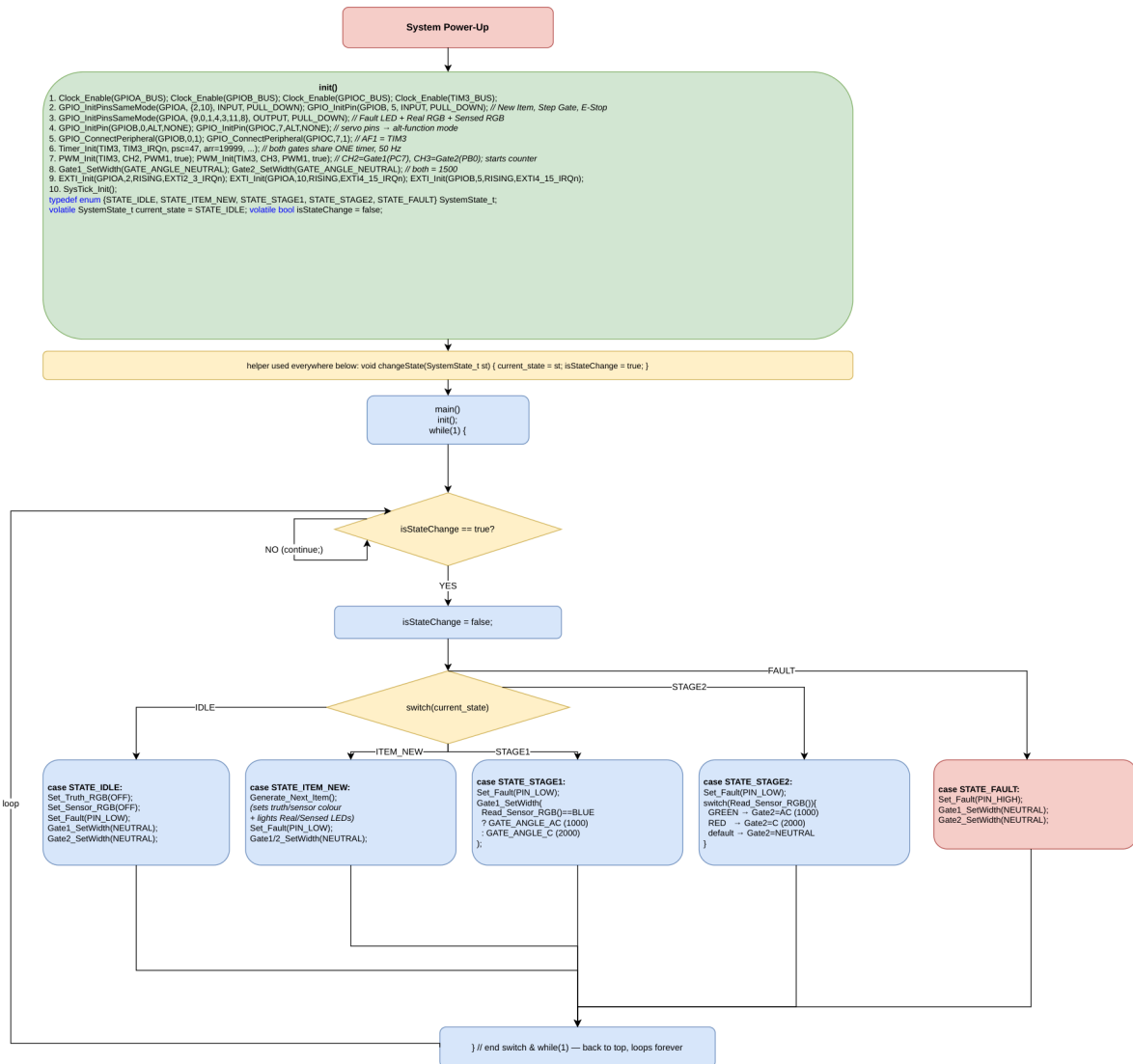


Figure 1 – Master flowchart (final.drawio.pdf): power-up, register initialisation and the main-loop FSM output logic

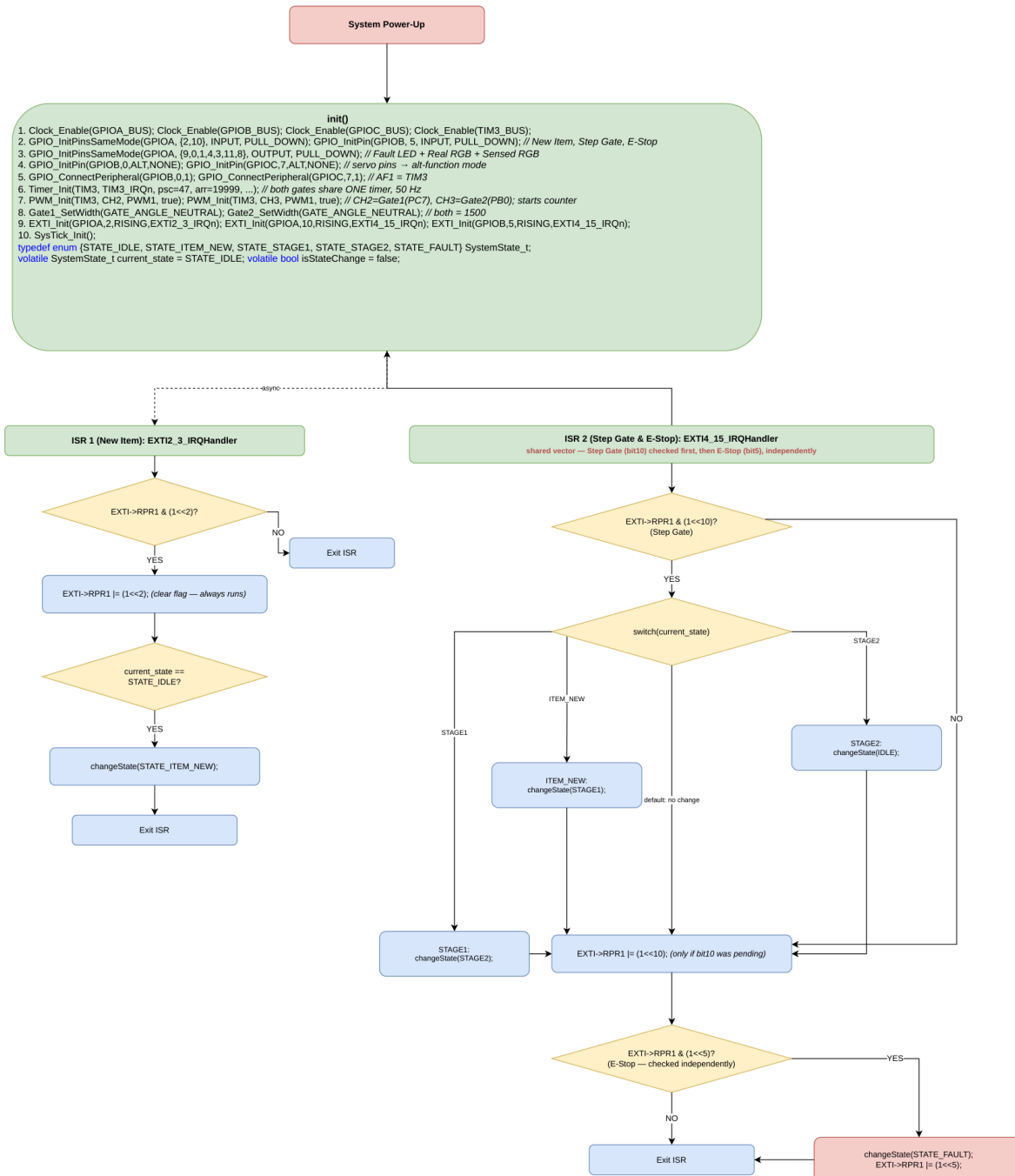


Figure 2 – ISR flowcharts (*interrupts.drawio.pdf*): EXTI2_3_IRQHandler (New Item) and EXTI4_15_IRQHandler (Step Gate / E-Stop)

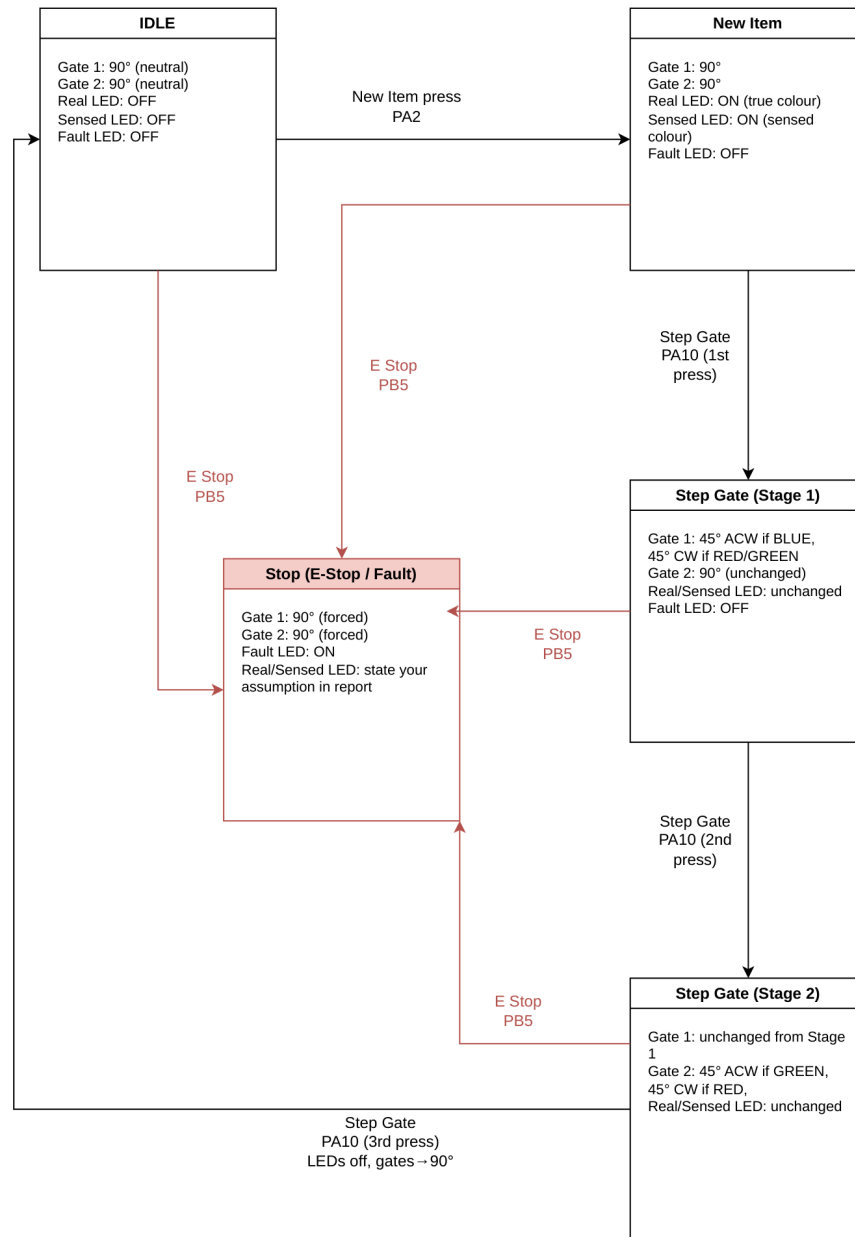


Figure 3 – FSM state diagram (state.drawio.pdf). During STOP (E-Stop/Fault), the Real and Sensed LEDs keep their last colour (Section 4.5).

6. Test Results

Method. The code was built in VS Code and run in the Wokwi emulator. For each item I pressed New Item and recorded the Real and Sensed LED colours. I then pressed Step Gate three times, recording the Gate 1 action (1st press) and Gate 2 action (2nd press), and checked that the 3rd press returned both gates to neutral and turned the LEDs off. I tested 30 items; the raw log is in docs/data.txt. Rows with a sensor fault (real $\neq$ sensed) are highlighted.

#	Real colour	Sensed colour	Sensor fault?	Gate 1 (1st Step)	Gate 2 (2nd Step)	Expected G1 / G2 (from sensed)	Item routed to	Gates correct for sensed colour?	Bin matches real colour?
1	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
2	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
3	Green	Blue	Yes	45° ACW	Neutral	ACW / N	Blue bin	✓	✗ (sensor fault)
4	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓
5	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
6	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
7	Red	Green	Yes	45° CW	45° ACW	CW / ACW	Green bin	✓	✗ (sensor fault)
8	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
9	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
10	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓
11	Green	Blue	Yes	45° ACW	Neutral	ACW / N	Blue bin	✓	✗ (sensor fault)
12	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
13	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
14	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓
15	Blue	Green	Yes	45° CW	45° ACW	CW / ACW	Green bin	✓	✗ (sensor fault)
16	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
17	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
18	Red	Blue	Yes	45° ACW	Neutral	ACW / N	Blue bin	✓	✗ (sensor fault)
19	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓
20	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
21	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
22	Green	Red	Yes	45° CW	45° CW	CW / CW	Red bin	✓	✗ (sensor fault)
23	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓
24	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
25	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
26	Blue	Red	Yes	45° CW	45° CW	CW / CW	Red bin	✓	✗ (sensor fault)
27	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓
28	Green	Green	No	45° CW	45° ACW	CW / ACW	Green bin	✓	✓
29	Blue	Blue	No	45° ACW	Neutral	ACW / N	Blue bin	✓	✓
30	Red	Red	No	45° CW	45° CW	CW / CW	Red bin	✓	✓

Table 3 – Classification results for 30 items (from docs/data.txt, numbered 1–30). ACW = anticlockwise, CW = clockwise, N = neutral.

Measure	Result
Items tested	30 (real colours: 11 blue, 10 green, 9 red)
Gate sequence correct for the sensed colour	30 / 30 (100%)
Item reached the bin of its real colour	23 / 30 (76.7%)
Sensor faults (real ≠ sensed)	7 / 30 (23.3%) – items 3, 7, 11, 15, 18, 22, 26
Blue-sensed items with Gate 2 left neutral	12 / 12
3rd Step press returned gates to 90° and LEDs off	30 / 30

Table 4 – Summary of results

The video (Video.mp4) shows items 1–5 of Table 3 being sorted, and the E-Stop turning on the Fault LED. Three more items were stepped through quickly at about 3:56–4:07 in the video but were not logged. Items 6–30 were tested in separate runs of the same code and are not in this recording.

7. Discussion of Correctness

The system is designed to route each item using the **sensed** colour. On that basis, all 30 items behaved as expected. Every sensed-blue item turned Gate 1 ACW and left Gate 2 neutral. Every sensed-green item gave Gate 1 CW then Gate 2 ACW. Every sensed-red item gave Gate 1 CW then Gate 2 CW. The 3rd Step press always reset the system.

Seven items (23.3%) went to the wrong bin compared with their real colour. This is expected and is not a code error: the provided *Apply_LFSR_Fault()* deliberately changes the detected colour about 20% of the time. For 30 items the expected number of faults is $30 \times 0.2 = 6$, with a standard deviation of $\sqrt{(30 \times 0.2 \times 0.8)} \approx 2.2$, so 7 faults is within normal random variation. In each fault case the gates correctly followed the (wrong) sensed colour, which is how the real system would behave.

8. Acknowledgement of Lecture and Provided Code

Most of the **stm_helper** library is based on code given in the ENGE X200 lectures and practicals, as listed below. My own additions are noted. The provided assignment code was copied into main.c as instructed.

Code	Source	My changes
Clock_Enable()	Lecture helper library (Timers lecture)	Added GPIOC_BUS and TIM3_BUS cases
GPIO_InitPin(), GPIO_OutputSetPin/ClearPin/TogglePin(), GPIO_InputReadPin()	Lecture helper library (GPIO / registers lectures)	None
Timer_Init()	Timers lecture	Added ARPE, UG (force update)

Code	Source	My changes
		and optional counter-enable parameters
EXTI_Init()	Lecture 3 Interrupts – EXTI_InitPin()	Takes a GPIO_TypeDef* and works out the port index itself
PWM_Init(), GPIO_ConnectPeripheral()	Register steps from the Capture/Compare & PWM lecture (MODER = AF, AFR, CCMR OCxM = 110, OCxPE, CCER)	Written as reusable functions for CH2/CH3
50 Hz servo set-up (PSC 47, ARR 19999) and 500–2500 μ s angle mapping	Capture/Compare & PWM lecture; Lecture 7 FSM servo example	Pulse widths set for $\pm 45^\circ$ gates
SysTick_Init(), Apply_LFSR_Fault(), Generate_Next_Item(), ItemColour_t	Provided assignment code (assignment_1.c/.h)	None

Table 5 – Code reuse and acknowledgement

9. Conclusion

The code configures GPIO, EXTI and TIM3 PWM at register level and uses a switch-case finite state machine, driven by the button interrupts, to control the two sorting gates. Testing on 30 items showed that the gates were correct for the sensed colour every time. The flowcharts, code, test log and video are included as listed in the appendix at the start of this report.